



Ứng dụng kiến trúc IoC/DI, GraalVM và Spring Boot trong phần mềm quản lý hiệu năng cao



```
render() {  
  return (  
    <React.Fragment>  
      <div className="py-5">  
        <div className="container">  
          <Title name="our" title="product">  
            <div className="row">  
              <ProductConsumer>  
                {(value) => {  
                  console.log(value)  
                }}  
              </ProductConsumer>  
            </div>  
          </div>  
        </div>  
      </React.Fragment>  
    )  
  )  
}
```



Presented By

NHÓM HMPP

Vấn đề:



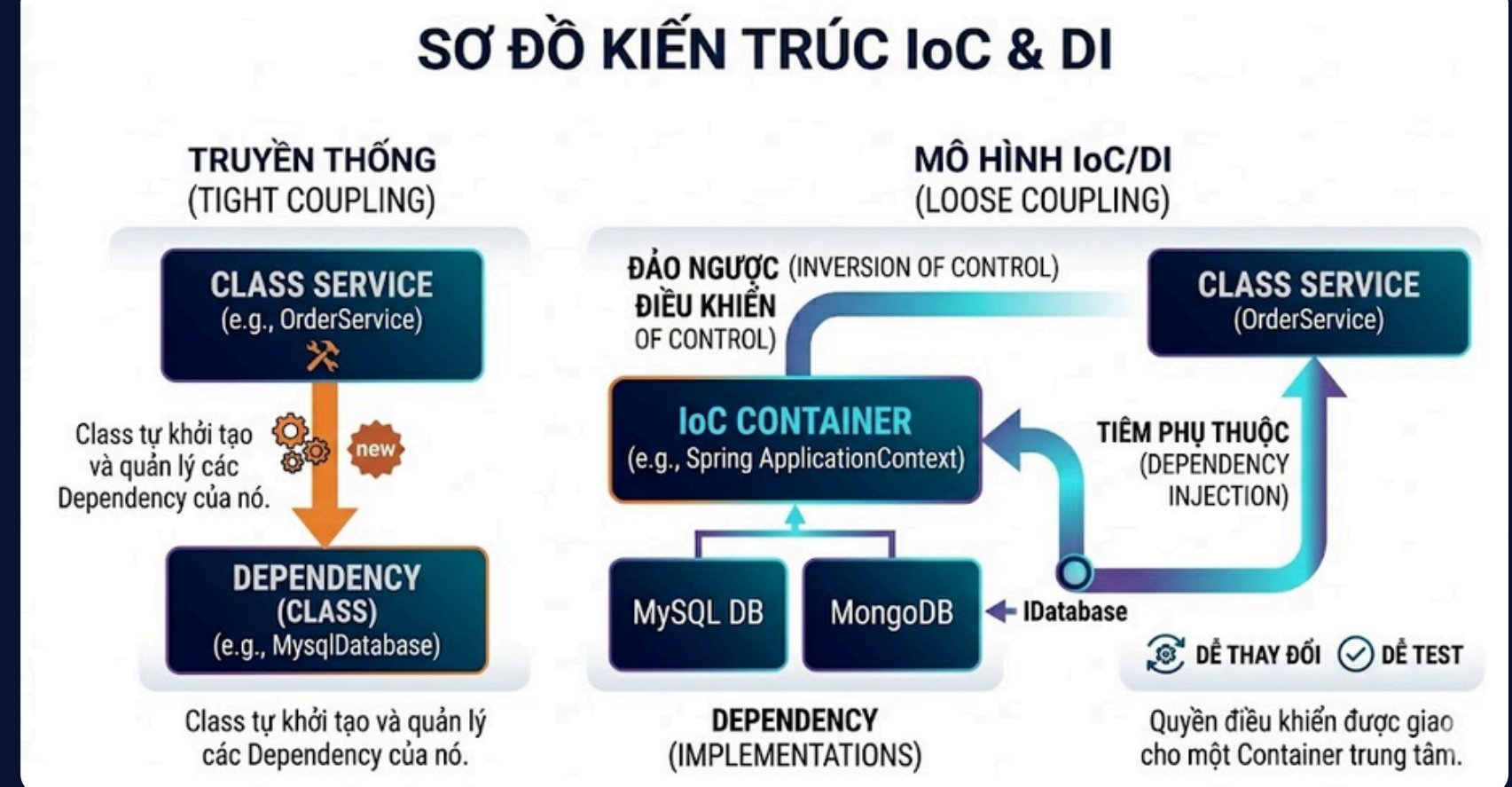
Cách phát triển Java truyền thống bộc lộ hạn chế về: Tight-Coupling (phụ thuộc chặt), tốn thời gian cấu hình thủ công và hiệu năng khởi động chậm.

Giải pháp:



IoC/DI, Spring Boot và GraalVM ra đời như một giải pháp toàn diện: Tối ưu hóa từ kiến trúc mã nguồn linh hoạt đến tốc độ thực thi Native Image đỉnh cao.

Khái niệm IoC/DI:



IoC (Inversion of Control - Đảo ngược điều khiển):

- Là nguyên lý thiết kế.
- Các thành phần không tự tạo tài nguyên.
- Giao toàn quyền quản lý cho Container trung tâm.



DI (Dependency Injection - Tiêm phụ thuộc):

- Là cách thức hoạt động (hiện thực hóa IoC).
- Container tự động "tiêm" (cung cấp) tài nguyên.
- Cấp phát chính xác vào đúng nơi cần thiết.

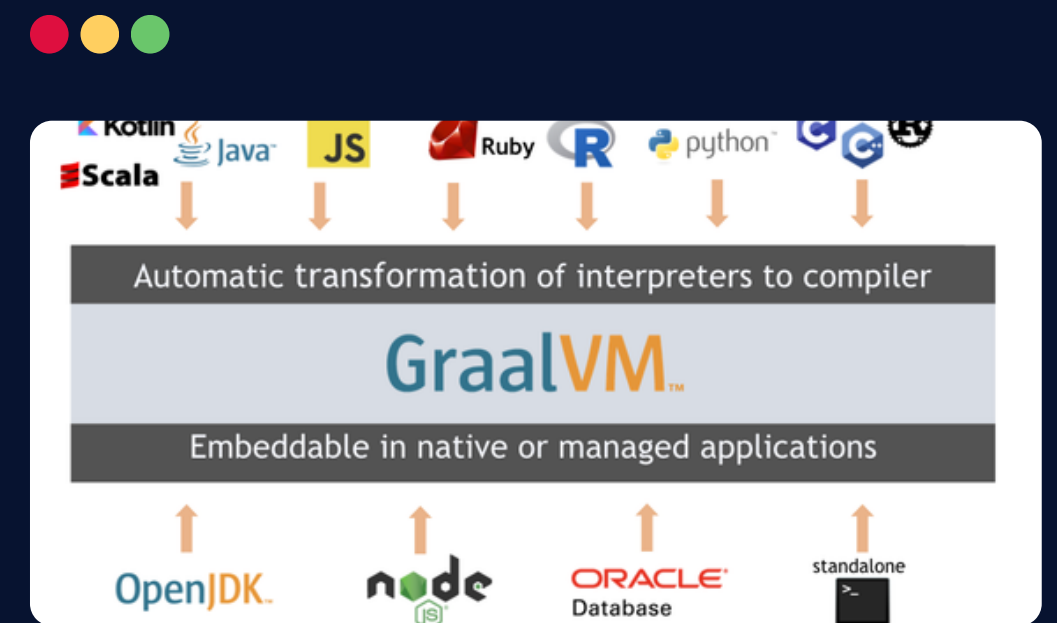


- Nền tảng: Mở rộng từ hệ sinh thái Spring Framework.
- Cơ chế cốt lõi: Auto-Configuration & Convention over Configuration.
- Đặc trưng kiến trúc: Tích hợp Embedded Server (Tomcat), tạo ứng dụng Stand-alone đạt chuẩn Production-ready.

SpringBoot Framework:



GraalVM



- JDK hiệu suất cao: Bộ công cụ Java thế hệ mới.
- Công nghệ AOT (Ahead-of-Time): Biên dịch mã trước khi chạy.
- Native Image: File thực thi độc lập (Không cần máy ảo JVM).
- Tối ưu phần cứng: Khởi động mili-giây, tiết kiệm RAM.



GrailVMTM



Nền tảng Runtime đa ngôn ngữ

1

**Tổng quan
về GraalVM**

2

**Java với
HotSpot JVM**

3

**Java với
HotSpot JVM**

4

**Cấu trúc
của GraalVM**

5

**Java với
GraalVM**

6

**Điểm tối ưu
của GraalVM**

Tổng quan về GraalVM



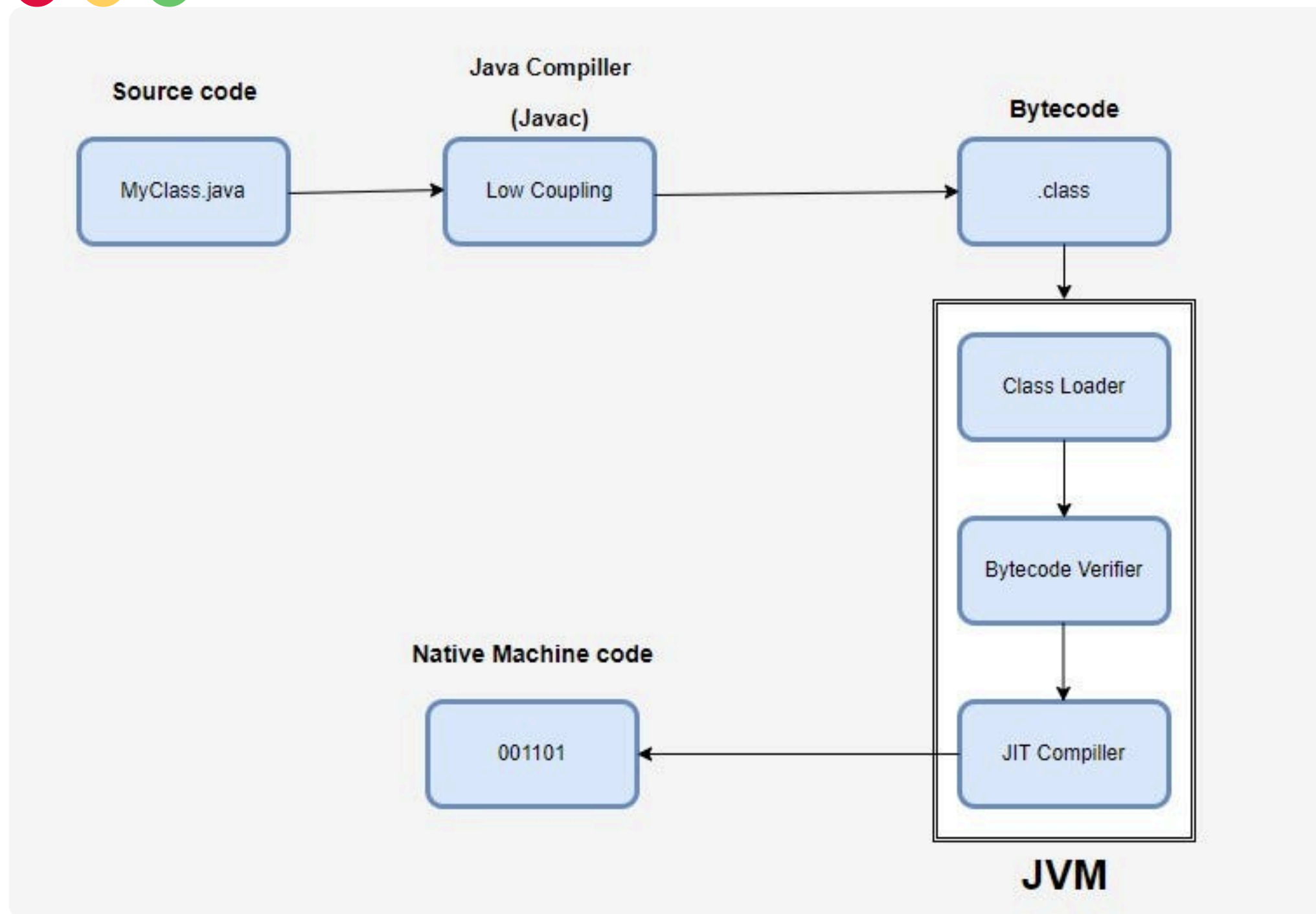
- Nền tảng Runtime hiệu suất cao
- Hỗ trợ JIT Compiler & AOT
- Polyglot
- Hỗ trợ đa nền tảng



Polyglot



Java với HotSpot



Trong nhiều năm HotSpot JVM là tiêu chuẩn vàng của Oracle

Quy trình:

.java → Bytecode → Class Loader → Interpreter + JIT (C1/C2) → Machine Code

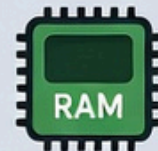
Hạn chế của HotSpot trong kỷ nguyên mới



Serverless & Microservices Require:



Ultra-fast startup

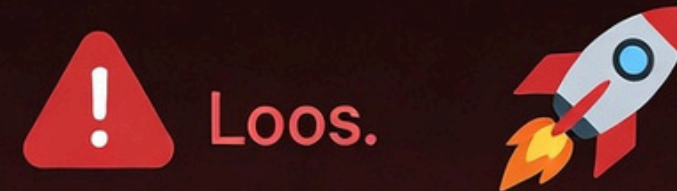


Low memory usage



Easy polyglot communication

HotSpot JVM Struggles With:



Loos.



Long startup time



High RAM usage



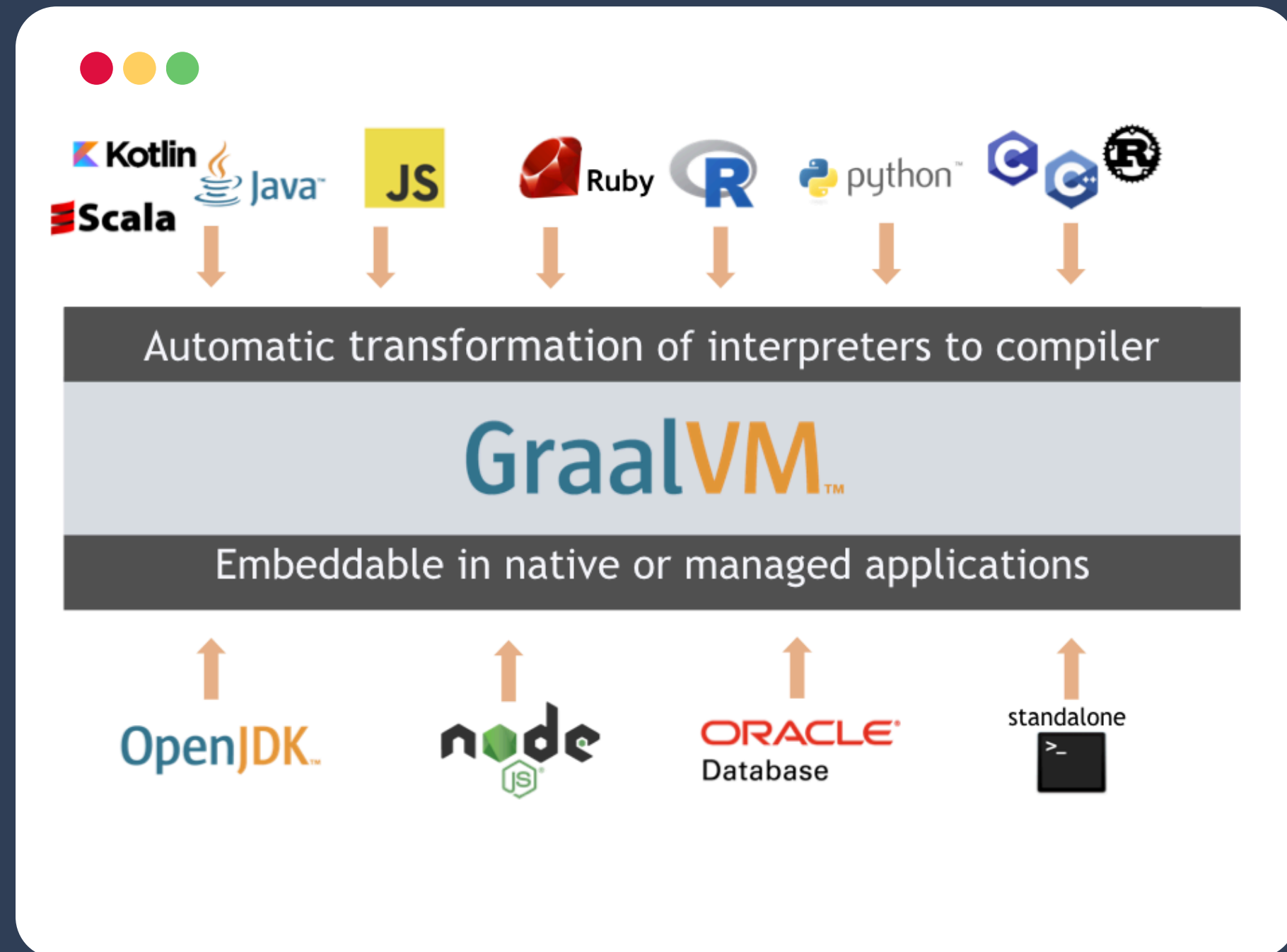
Serverless & Microservices đòi hỏi:

- Khởi động nhanh
- Bộ nhớ thấp
- Tương tác đa ngôn ngữ dễ dàng

→ HotSpot dần bộc lộ giới hạn
(startup chậm, RAM cao, polyglot khó)

Cấu trúc của GraalVM

- Graal compiler
 - Trình biên dịch JIT
 - Biên dịch bytecode, JVM → mã máy
- GraalVM Runtime
 - Gồm nhiều runtime khác nhau
 - Tích hợp liền mạch giữa các ngôn ngữ
- Native Image
 - công cụ tạo file thực thi



Java với GraalVM

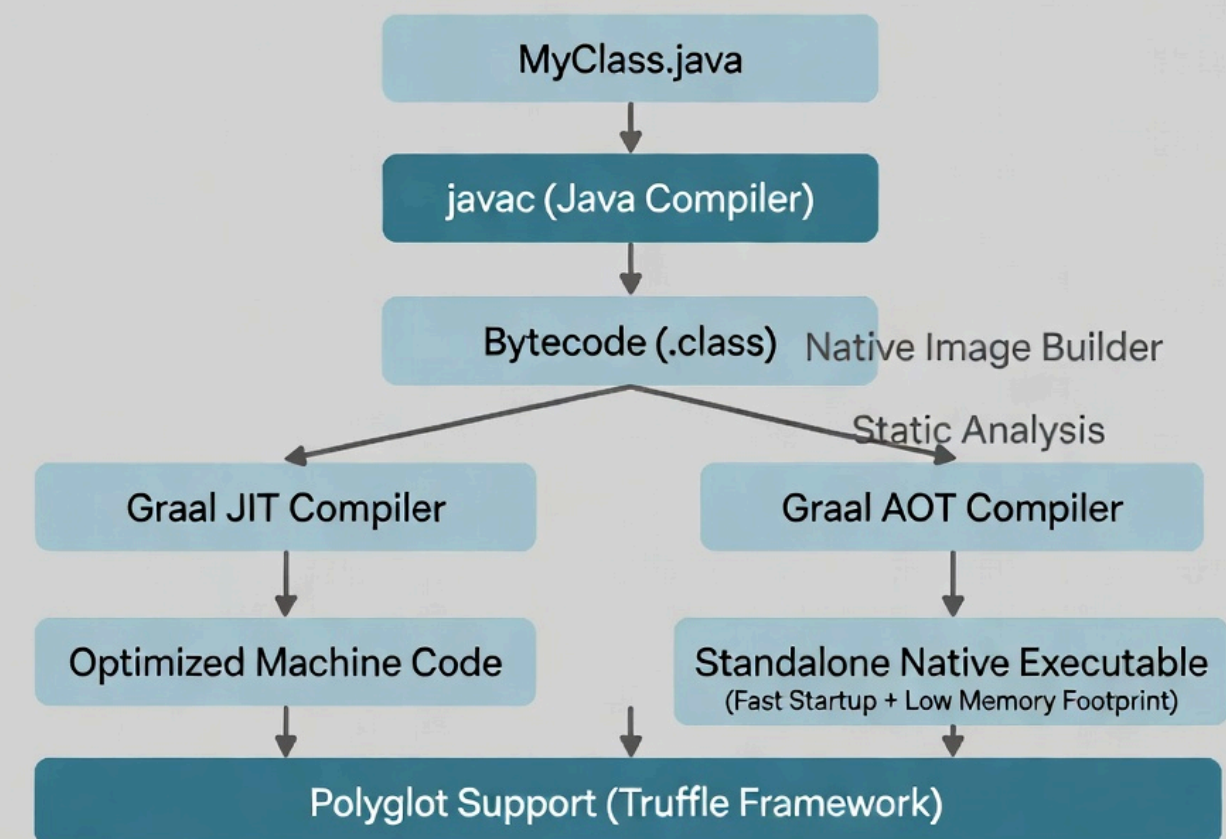


Cách 1: Chạy JIT với Graal Compiler

Cách 2: Native Image



Java after GraalVM Integration



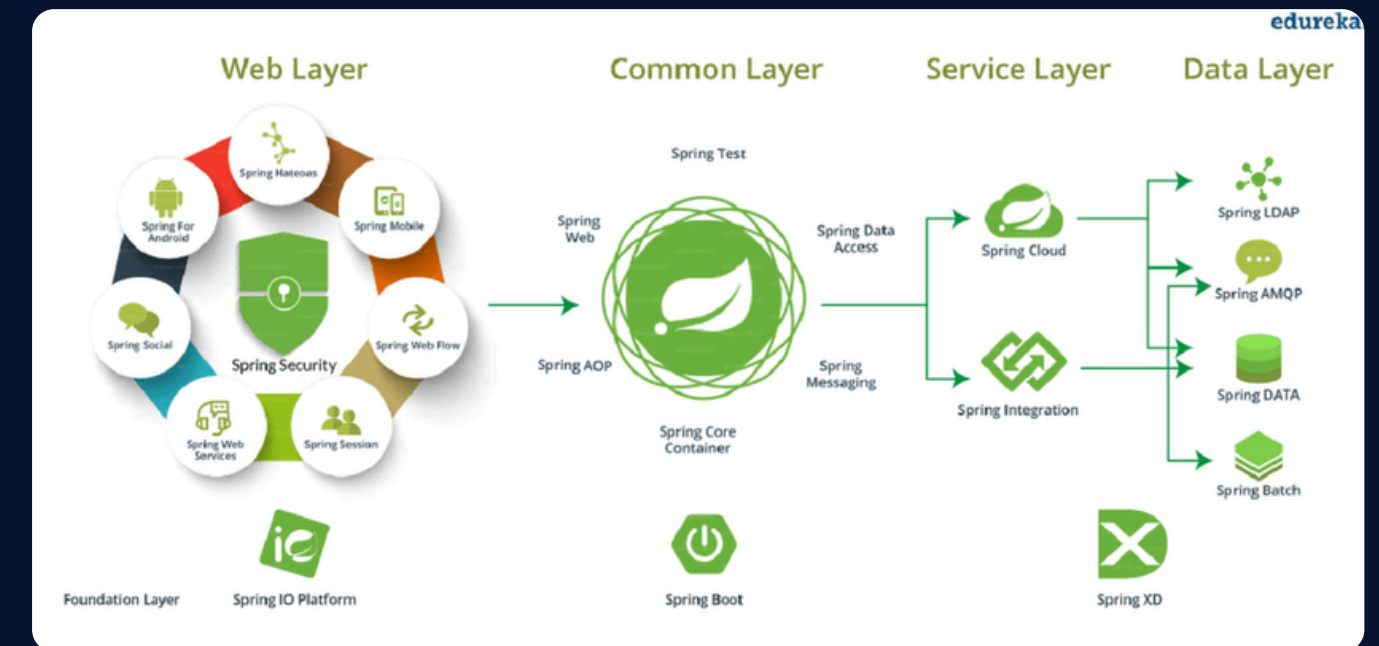
Điểm tối ưu của GraalVM



- Hỗ trợ đa ngôn ngữ
- Native Image
- Trình biên dịch Graal JIT
- Dễ tích hợp
- Truffle Framework
- Giảm chi phí startup, bộ nhớ



SPRINGBOOT framework



What is
Spring Boot
Framework?

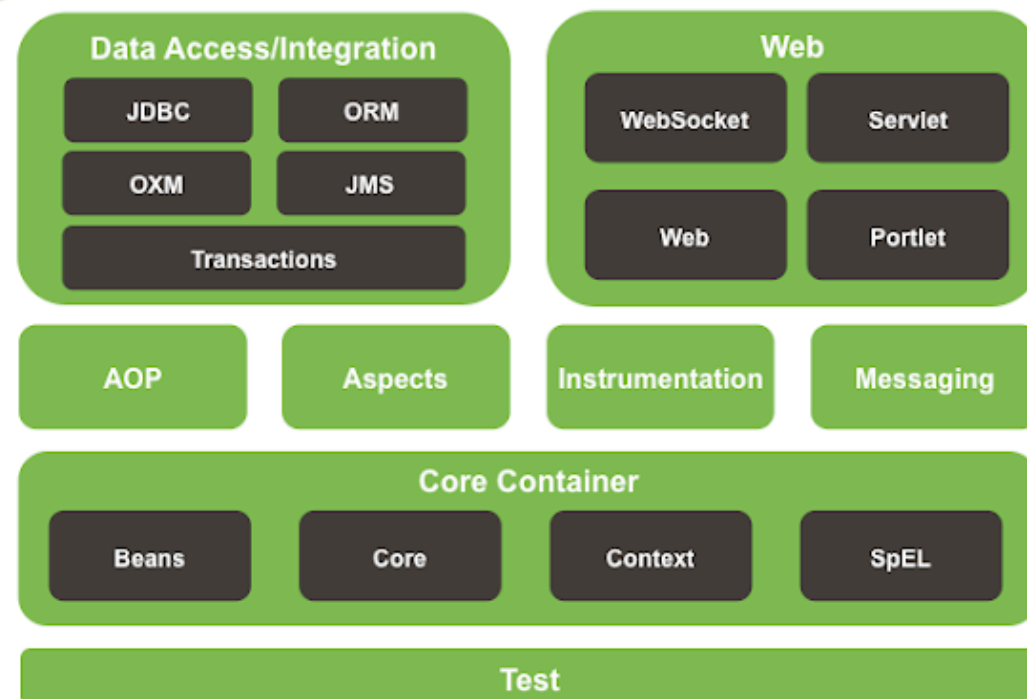


a enhanced framework

SPRING framework

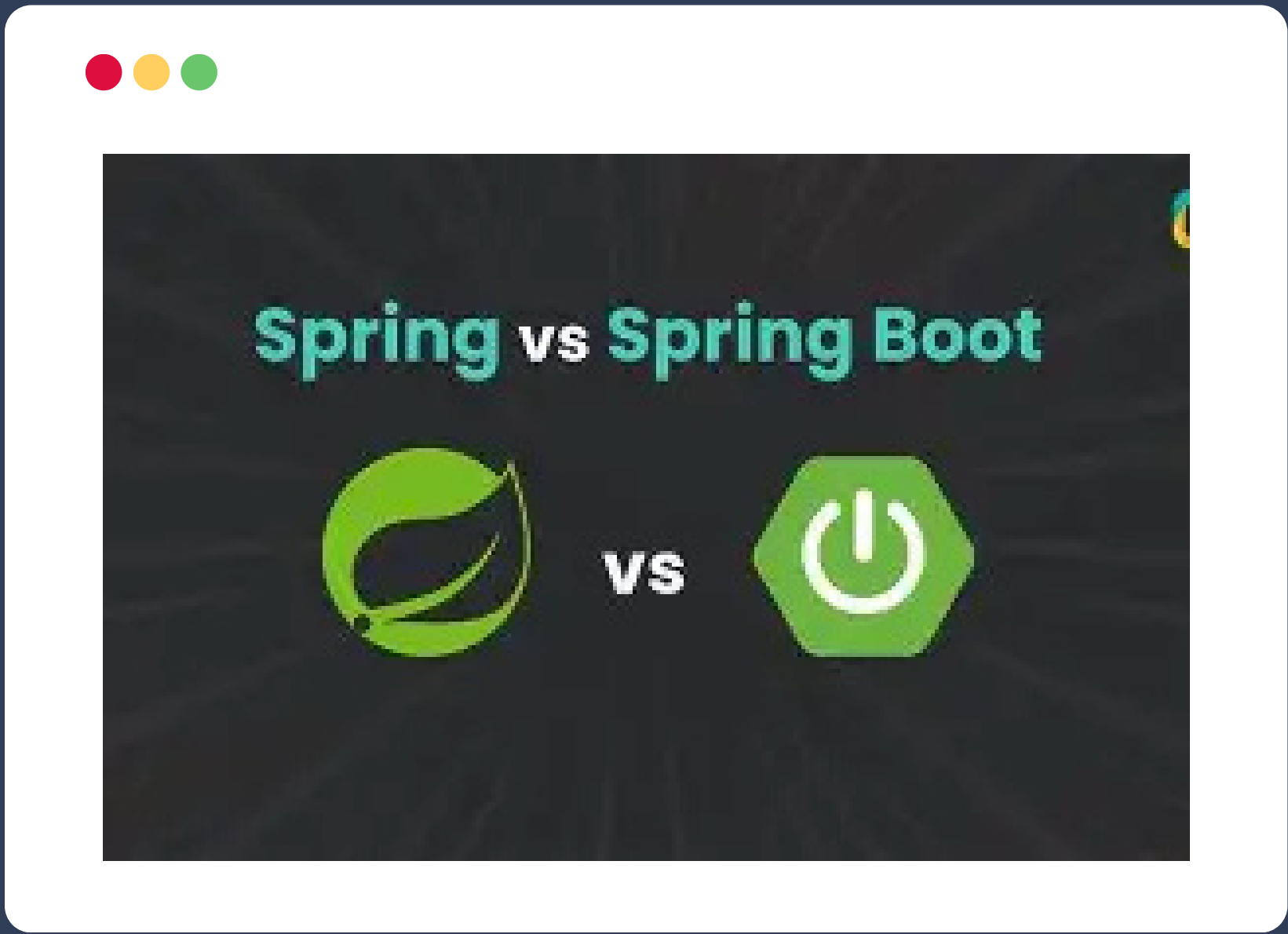


Spring Framework Runtime



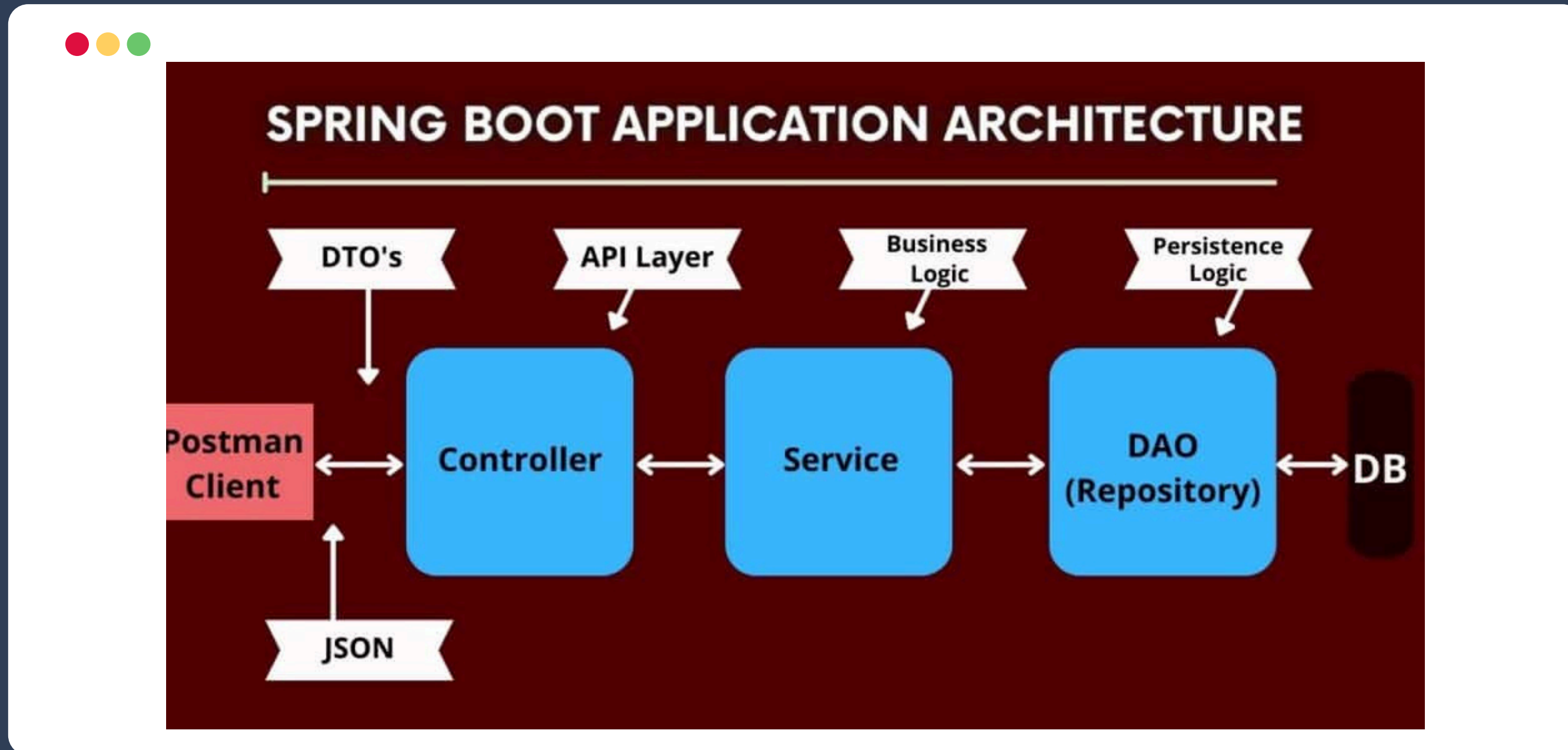
- Core Container: Nền tảng cốt lõi. Cung cấp tính năng IoC (Inversion of Control), DI (Dependency Injection) và Expression Language.
- Data Access / Integration: Quản lý giao tiếp Database (hỗ trợ JDBC, ORM, JMS và Transaction).
- Web (Spring MVC): Cung cấp bộ công cụ để phát triển các ứng dụng Web.
- AOP & Instrumentation: Hỗ trợ Lập trình hướng khía cạnh (Aspect-Oriented Programming).
- Test: Hỗ trợ kiểm thử ứng dụng dễ dàng với JUnit và TestNG.

Spring & Springboot

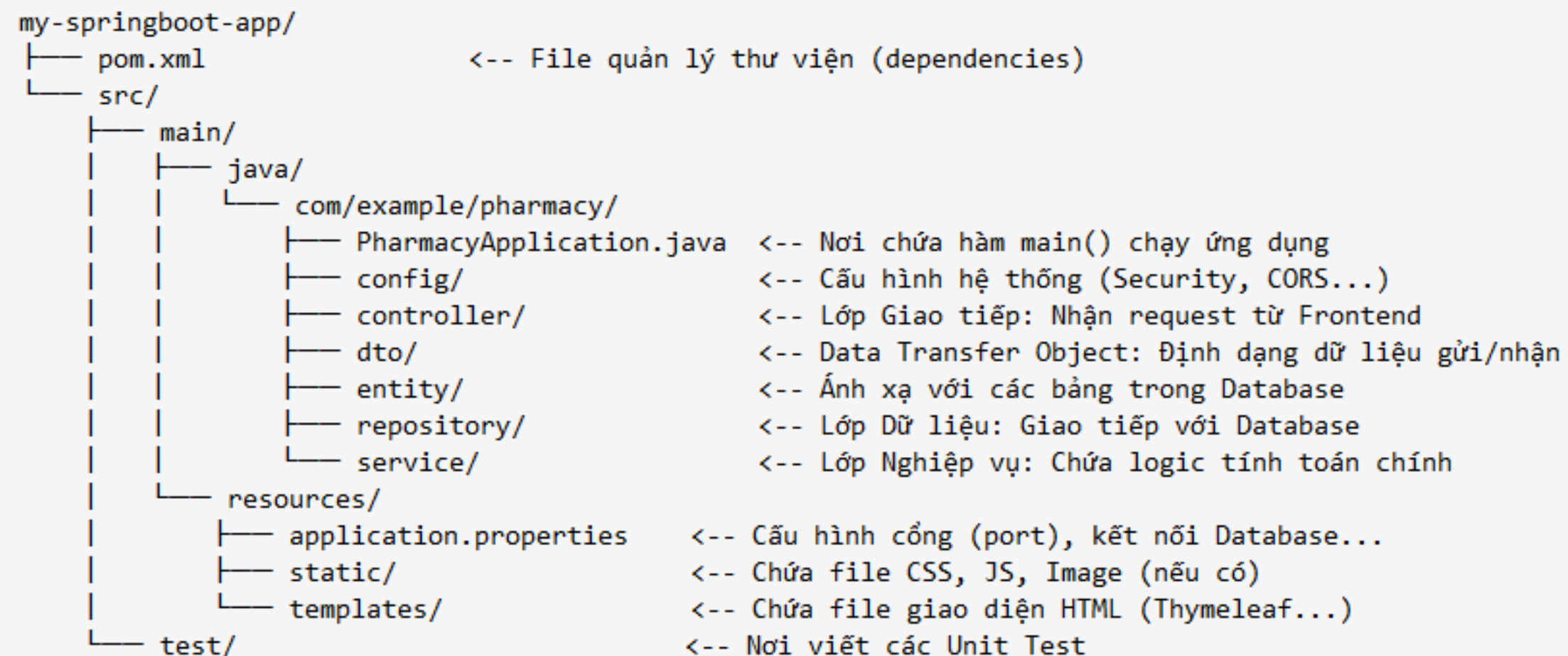
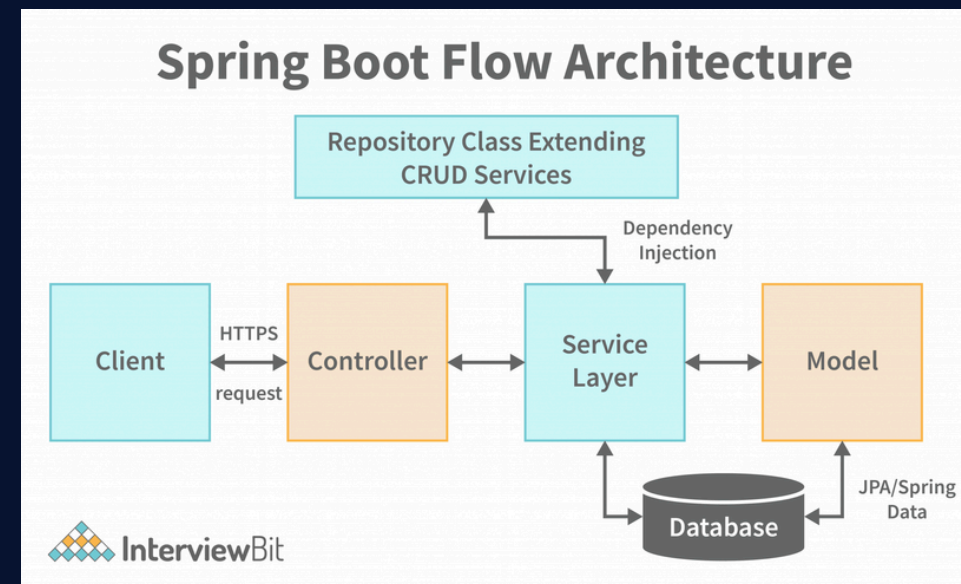


	Spring Framework	Spring Boot
Mục tiêu chính	Cung cấp bộ khung toàn diện cho ứng dụng Java Enterprise	Đơn giản hóa và tăng tốc quá trình phát triển ứng dụng Spring
Cấu hình	Thủ công, phức tạp (phải tự setup qua XML hoặc Annotations)	Tự động (Auto-configuration) theo triết lý "Convention over Configuration"
Quản lý Thư viện	Tự quản lý từng dependency và xử lý xung đột phiên bản	Quản lý tập trung qua Starters (VD: spring-boot-starter-web).
Máy chủ (Server)	Đóng gói .war, phải tự deploy lên server độc lập (Tomcat, WebLogic...)	Tích hợp sẵn (Embedded Server), đóng gói .jar và chạy độc lập

Cấu trúc của Springboot



Cấu trúc của Springboot



IoC & DI



- **IoC (Nguyên lý thiết kế):** Đảo ngược quyền điều khiển. Giao việc khởi tạo và quản lý đối tượng cho một bộ chứa (Container) trung tâm thay vì class tự dùng từ khóa new.
- **DI (Kỹ thuật thực hiện):** Cơ chế để Container tự động "tiêm" (cung cấp) các đối tượng phụ thuộc vào đúng class đang cần chúng.
- **Mục tiêu cốt lõi:** Loại bỏ sự phụ thuộc cứng, đạt được kiến trúc Khớp nối lỏng (Loosely Coupled).



Spring IoC Container



Bản chất - Core Concept

Là thành phần nền tảng (đại diện bởi `ApplicationContext`) chịu trách nhiệm khởi tạo, cấu hình và quản lý vòng đời của các đối tượng (Spring Beans).



Đảo ngược điều khiển - Inversion of Control

Chuyển giao hoàn toàn quyền kiểm soát luồng thực thi và khởi tạo đối tượng từ lập trình viên sang Spring Framework.



Quy trình xử lý - Bean Lifecycle:

1. **Component Scanning:** Tự động quét classpath để phát hiện các class được cấu hình metadata (`@Service`, `@Repository`...).
2. **Instantiation (Khởi tạo):** Đưa các đối tượng vào bộ nhớ của Container
3. **Dependency Injection (Tiêm phụ thuộc):** Tự động phân tích cây phụ thuộc và tiêm các Bean cần thiết vào nhau một cách toàn vẹn.



Giá trị mang lại

Đảm bảo tính nhất quán (Consistency), tập trung hóa quản lý bộ nhớ và thực thi triệt để kiến trúc Khớp nối lỏng (Loosely Coupled).

Phương pháp Tiêm Dependency Injection (DI)



3. Field Injection

Dependency được tiêm trực tiếp vào các trường (field) thông qua `@Autowired`. Cách đơn giản nhất để viết nhưng ít được khuyến nghị nhất vì khó viết test, coupling cao.

```
@Autowired  
private AccountRepository accountRepo;
```



1. Constructor Injection

Dependency được tiêm qua Constructor của class. Đây là cách khuyến nghị nhất vì đảm bảo các dependency không null khi object được tạo.

```
public AccountServiceImpl(AccountRepository  
accountRepo) {  
    this.accountRepo = accountRepo;  
}
```



2. Setter Injection

Dependency được tiêm qua các hàm setter public. Cho phép thay đổi dependency sau khi object được tạo. Phù hợp cho dependency không bắt buộc.

```
@Autowired  
public void  
setAccountRepository(AccountRepository  
accountRepo) {  
    this.accountRepo = accountRepo;  
}
```

IoC/DI & Spring Boot

- Sử dụng @Service để Spring IoC Container quản lý và tiêm phụ thuộc (DI) qua Constructor.
- Định nghĩa Interface AccountService

```
AccountService.java X AccountServiceImpl.java
src_APP > backend > src > main > java > com > pharmacy > backend > service > AccountService.java > ...
1 package com.pharmacy.backend.service;
2 import com.pharmacy.backend.dto.LoginResponse;
3
4 public interface AccountService {
5
6     LoginResponse checkLogin(String sdt, String password);
7
8     boolean createStaffAccount(String sdt, String rawPassword, String vaiTro);
9
10    boolean changePassword(String sdt, String newPassword);
11 }
12 |
```

- Interface chỉ khai báo Method Signature
- Cài đặt AccountServiceImpl kế thừa và @Override để viết logic thực tế.
- Tiêm phụ thuộc (DI): Sử dụng Constructor Injection

=> Hệ thống đạt được kiến trúc **Loosely Coupled** bằng kỹ thuật **DI**, Các class chỉ phụ thuộc vào Interface, cho phép Spring tự động xử lý và ghép nối các implementation cụ thể

```
AccountService.java M AccountServiceImpl.java M X
src_APP > backend > src > main > java > com > pharmacy > backend > service > AccountServiceImpl.java > ...
1 package com.pharmacy.backend.service;
2
3 > import java.util.Optional; ...
13
14 @Service
15 public class AccountServiceImpl implements AccountService {
16
17     private final AccountRepository accountRepo;
18     private final PasswordEncoder passwordEncoder;
19     private final JwtUtils jwtUtils;
20
21     public AccountServiceImpl(AccountRepository accountRepo, PasswordEncoder passwordEncoder, JwtUtils jwtUtils) {
22         this.accountRepo = accountRepo;
23         this.passwordEncoder = passwordEncoder;
24         this.jwtUtils = jwtUtils;
25     }
26
27     @Override
28 > public LoginResponse checkLogin(String sdt, String password) { ...
47
48     @Override
49 > public boolean createStaffAccount(String sdt, String rawPassword, String vaiTro) { ...
70
71     @Override
72 > public boolean changePassword(String sdt, String newPassword) { ...
86
87 }
88 |
```

●●● Code không sử dụng IOC/DI

```
package com.pharmacy.backend.service;

import com.pharmacy.backend.repository.AccountRepository; // Phải import class cụ thể
import com.pharmacy.backend.security.JwtUtils;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder; // Phải gọi thư viện cụ thể
//...

public class AccountServiceImpl implements AccountService {

    private final AccountRepository accountRepo;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtils jwtUtils;

    public AccountServiceImpl() {
        // KHÔNG DÙNG DI: Tự khởi tạo các đối tượng bằng từ khóa 'new'
        this.accountRepo = new AccountRepository();
        this.passwordEncoder = new BCryptPasswordEncoder();
        this.jwtUtils = new JwtUtils();
    }

    @Override
    public LoginResponse checkLogin(String sdt, String password) {
        // ...
    }

    // ... các method khác
}
```

- Spring Container tự tiêm phụ thuộc => Khớp nối lỏng (Loosely Coupled).
- Dễ bảo trì & Mở rộng: Linh hoạt thay đổi logic mà không cần sửa code class này.
- Tối ưu Unit Test: Dễ dàng truyền đối tượng giả (Mock) thông qua Constructor.

- Tự khởi tạo bằng từ khóa new => Khớp nối chặt (Tightly Coupled).
- Khó bảo trì: Class bị gắn cố định với một đối tượng cài đặt cụ thể.
- Khó viết Unit Test: Không thể tách rời để test bằng các đối tượng giả (Mock).

●●● Code sử dụng IOC/DI

```
J AccountService.java M  J AccountServiceImpl.java M X
src_APP > backend > src > main > java > com > pharmacy > backend > service > J AccountServiceImpl.java > ...
1  package com.pharmacy.backend.service;
2
3  > import java.util.Optional; ...
13
14  @Service
15  public class AccountServiceImpl implements AccountService {
16
17      private final AccountRepository accountRepo;
18      private final PasswordEncoder passwordEncoder;
19      private final JwtUtils jwtUtils;
20
21      public AccountServiceImpl(AccountRepository accountRepo, PasswordEncoder passwordEncoder, JwtUtils jwtUtils) {
22          this.accountRepo = accountRepo;
23          this.passwordEncoder = passwordEncoder;
24          this.jwtUtils = jwtUtils;
25      }
26
27      @Override
28  > public LoginResponse checkLogin(String sdt, String password) { ...
47
48      @Override
49  > public boolean createStaffAccount(String sdt, String rawPassword, String vaiTro) { ...
70
71      @Override
72  > public boolean changePassword(String sdt, String newPassword) { ...
86
87  }
88 |
```

Tổng kết

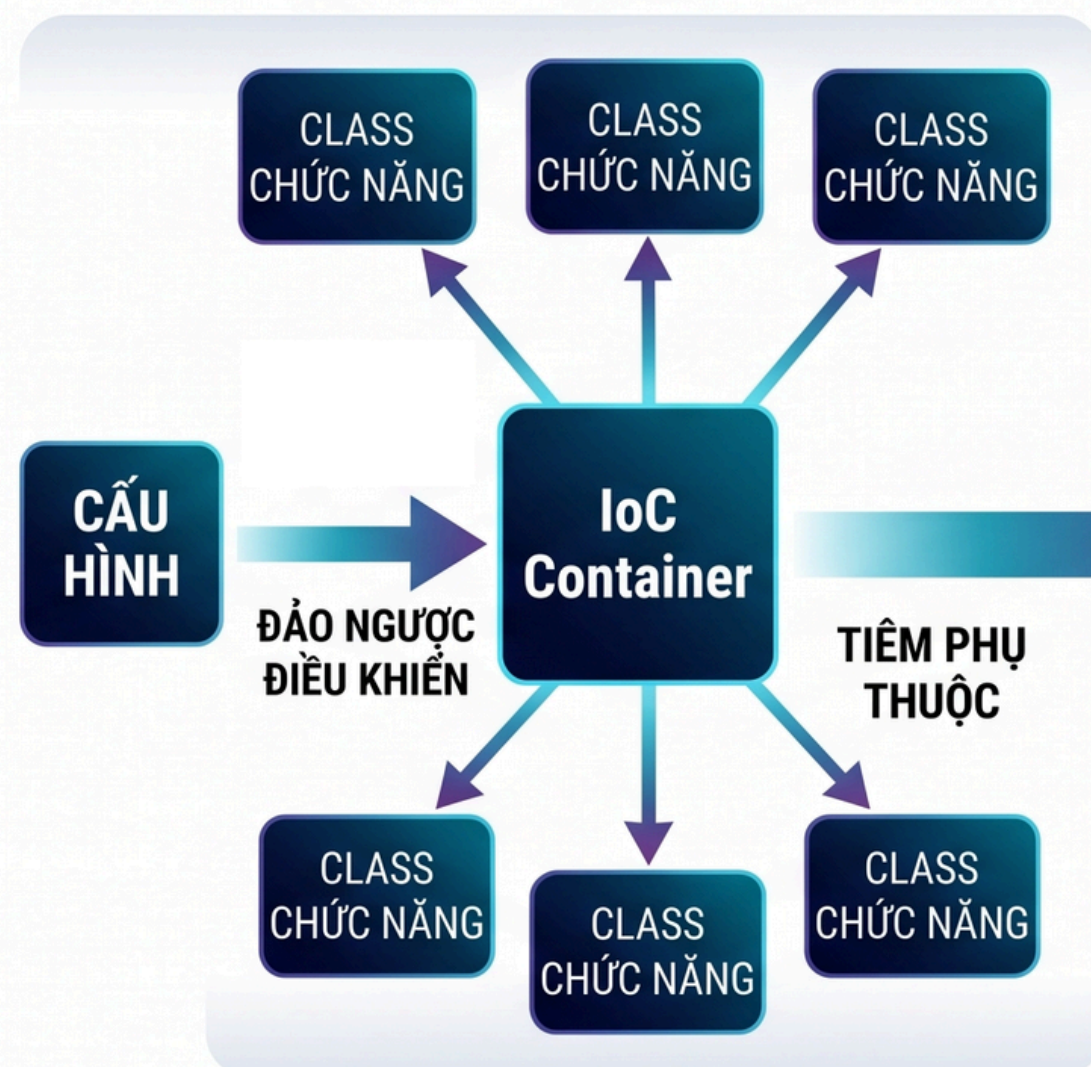


- IoC/DI: Tối ưu Kiến trúc => Code sạch, dễ bảo trì và mở rộng.
- Spring Boot: Tối ưu Phát triển => Xây dựng hệ thống nhanh, chuẩn Production.
- GraalVM: Tối ưu Vận hành => Khởi động cực nhanh, siêu tiết kiệm RAM.

TỐI ƯU HÓA HIỆU NĂNG ỨNG DỤNG JAVA

KIẾN TRÚC IoC & DI

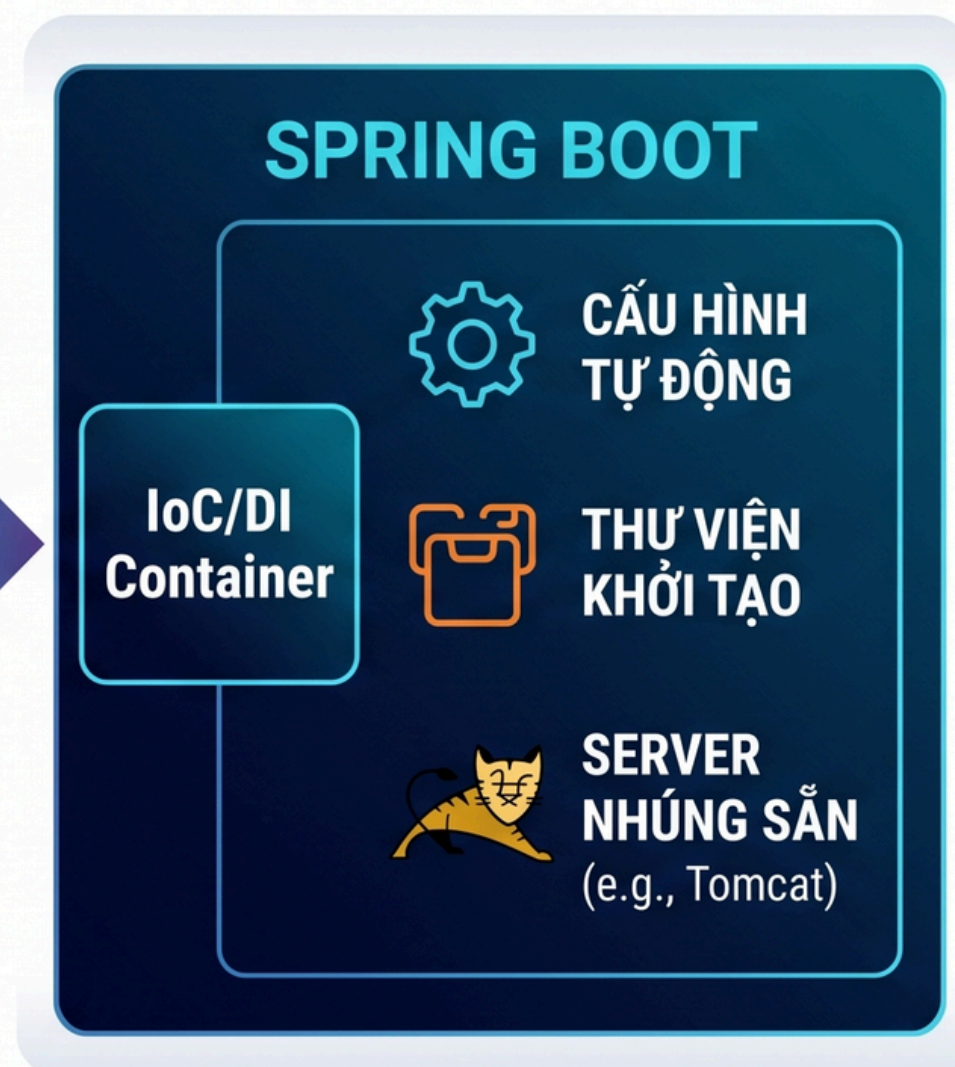
Giảm phụ thuộc chặt chẽ, dễ bảo trì



KIẾN TRÚC IoC & DI

FRAMEWORK SPRING BOOT

Phát triển nhanh, Production-ready



JAVA
BYTECODE
(.jar)

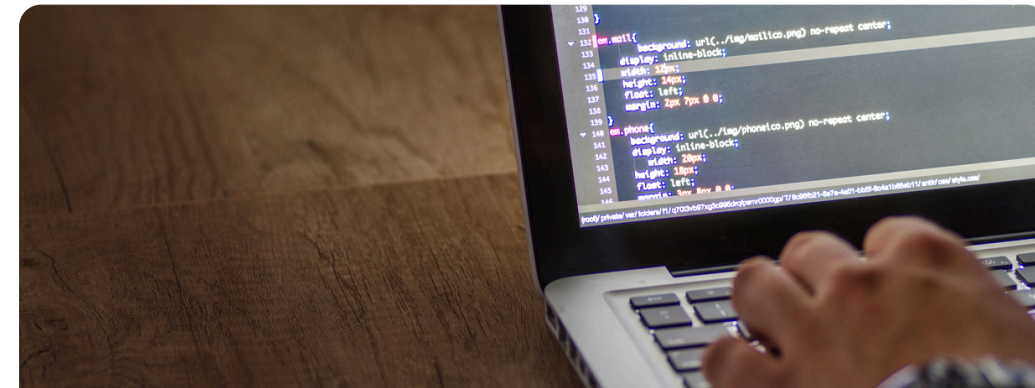
BUILD
NATIVE

NỀN TẢNG **GRAALVM**

Biên dịch Native Image bản địa



ỨNG DỤNG NATIVE
HIỆU NĂNG CAO



Thank you
for